Zewail City University of Science and Technology

School of Engineering

Robotic Arm Project

Course Code: SPC 428

Submitted by:

Asmaa Sapry 202201338
Raed Asham 202200272

Instructor:

Dr. Mohanned Draz

January 5, 2026

1 Introduction

Forward kinematics (FK) is the process of determining the position and orientation of a robot end-effector given the joint variables. This report derives the forward kinematics of a robotic arm by following a matrix-based approach. The derivation starts from basic rotation and translation matrices, proceeds to the 2D case, and then extends to a 3D robotic manipulator, exactly following the handwritten derivation.

2 Basic Transformation Matrices

2.1 Rotation Matrices

Rotation about the x-axis:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha & 0 \\ 0 & \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about the y-axis:

$$R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\beta & 0 & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotation about the z-axis:

$$R_z(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 & 0 \\ \sin\gamma & \cos\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Translation Matrices

$$T_x(a) = \begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_y(b) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_z(c) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3 Homogeneous Transformation

Each link transformation can be written as:

$$A_i = \begin{bmatrix} R_i & d_i \\ 0 & 1 \end{bmatrix}$$

4 Forward Kinematics: 2D Case

For the 2D robotic arm, all rotations occur about the z-axis.

4.1 Link Transformations

$$A_1 = \begin{bmatrix} c_1 & -s_1 & 0 & 0 \\ s_1 & c_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0 \\ s_2 & c_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.2 Overall Transformation

$$T_2^0 = A_1 A_2$$

$$T_2^0 = \begin{bmatrix} c_1c_2 - s_1s_2 & -c_1s_2 - s_1c_2 & 0 & 0 \\ s_1c_2 + c_1s_2 & -s_1s_2 + c_1c_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5 Forward Kinematics: 3D Case

5.1 Transformation Matrices

$$A_1 = \begin{bmatrix} c_1 & 0 & s_1 & 0 \\ s_1 & 0 & -c_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_2 = \begin{bmatrix} c_2 & -s_2 & 0 & 0 \\ s_2 & c_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} c_3 & -s_3 & 0 & 0 \\ s_3 & c_3 & 0 & 0 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

5.2 Overall Transformation

$$T_3^0 = A_1 A_2 A_3$$

$$T_3^0 = \begin{bmatrix} c_1 s_2 s_3 + c_1 c_2 c_3 & -c_1 c_2 s_3 - c_1 s_2 c_3 & s_1 & 0 \\ -s_1 s_2 s_3 + s_1 c_2 c_3 & -s_1 c_2 s_3 - s_1 s_2 c_3 & -c_1 & 0 \\ c_2 s_3 + s_2 c_3 & -s_2 s_3 + c_2 c_3 & 0 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $c_i = \cos\theta_i$ and $s_i = \sin\theta_i$.

6 Inverse Kinematics Analysis

Inverse kinematics (IK) determines the joint variables that achieve a desired end-effector position. In this section, the IK is derived first for a 2D planar arm, then modified to the 3D case following the trigonometric relations shown in the provided derivation.

6.1 Inverse Kinematics for a 2D Planar Arm (2-DOF)

Consider a 2-link planar arm with link lengths L_1 and L_2 and desired end-effector position (x, y). Define the planar distance from the base to the end-effector as:

$$r = \sqrt{x^2 + y^2}$$

Using the cosine rule on the triangle formed by L_1, L_2, and r:

$$\cos\theta_2 = \frac{r^2 - L_1^2 - L_2^2}{2L_1 L_2}$$

Hence, the second joint angle has two possible solutions (elbow-up / elbow-down):

$$\theta_2 = \pm \cos^{-1}\left(\frac{r^2 - L_1^2 - L_2^2}{2L_1 L_2}\right)$$

To compute θ_1, define the auxiliary angle:

$$\phi = \text{atan2}(y, x)$$

and

$$\psi = \text{atan2}\left(L_2 \sin\theta_2,\ L_1 + L_2 \cos\theta_2\right)$$

Then:

$$\theta_1 = \phi - \psi$$

Remark: The $\pm$ in θ_2 generates the two IK configurations. The valid solution is selected based on joint limits and the required posture.

6.2 Modification to 3D (3-DOF RRR) to Find Joint Angles

For the 3D arm, the desired end-effector position is (x, y, z). First, define:

$$k = \sqrt{x^2 + y^2}$$

and the spatial distance:

$$r = \sqrt{k^2 + z^2} = \sqrt{x^2 + y^2 + z^2}$$

which matches the relation shown in Eq. (12).

6.2.1 Angle θ_1

From the projection in the x-y plane, the base rotation angle is:

$$\theta_1 = \text{atan2}(y, x)$$

(this corresponds to Eq. (13)).

6.2.2 Angle θ_3 (Cosine Rule)

Applying the cosine rule in 3D using the triangle formed by L_1, L_2, and r:

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2\cos(180^\circ - \theta_3)$$

which leads to the expression shown in the provided derivation:

$$\theta_3 = -\cos^{-1}\left(\frac{x^2 + y^2 + z^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

(this corresponds to Eq. (15)).

6.2.3 Angle θ_2 (Using α and β)

From trigonometry as in Eqs. (16) and (17), define:

$$\alpha = \sin^{-1}\left(\frac{z}{r}\right)$$

$$\beta = \tan^{-1}\left(\frac{L_2\sin(\theta_3)}{L_1 + L_2\cos(\theta_3)}\right)$$

Therefore, the second joint angle is:

$$\theta_2 = \sin^{-1}\left(\frac{z}{r}\right) + \tan^{-1}\left(\frac{L_2\sin(\theta_3)}{L_1 + L_2\cos(\theta_3)}\right)$$

which matches Eq. (18) in the provided figures.

Remark: In implementation, it is recommended to compute the second term using:

$$\beta = \text{atan2}\left(L_2\sin\theta_3,\ L_1 + L_2\cos\theta_3\right)$$

to ensure numerical stability and correct quadrant selection.

7 3D CAD Design of the Robotic Arm

To validate the kinematic model and visualize the mechanical structure, a 3D CAD model of the robotic arm was developed. The CAD design represents the physical realization of the derived kinematic parameters, including link lengths, joint orientations, and degrees of freedom.

The model consists of three revolute joints corresponding to the analytical model derived in the forward and inverse kinematics sections. Each link was designed with appropriate dimensions to match the parameters L_1, L_2, and the offset along the z-axis. The joints are aligned to ensure consistency between the CAD model and the mathematical reference frames used in the kinematic analysis.

The CAD model serves several purposes:

- Verifying the correctness of the kinematic assumptions.
- Visualizing joint motions and workspace.
- Providing a basis for future dynamic analysis and control implementation.
- Enabling potential fabrication or simulation in multibody software.

7.1 CAD Model Representation

Figure 5 shows the 3D CAD model of the robotic arm. The figure illustrates the base, links, and joints, highlighting the overall geometry and joint configuration.

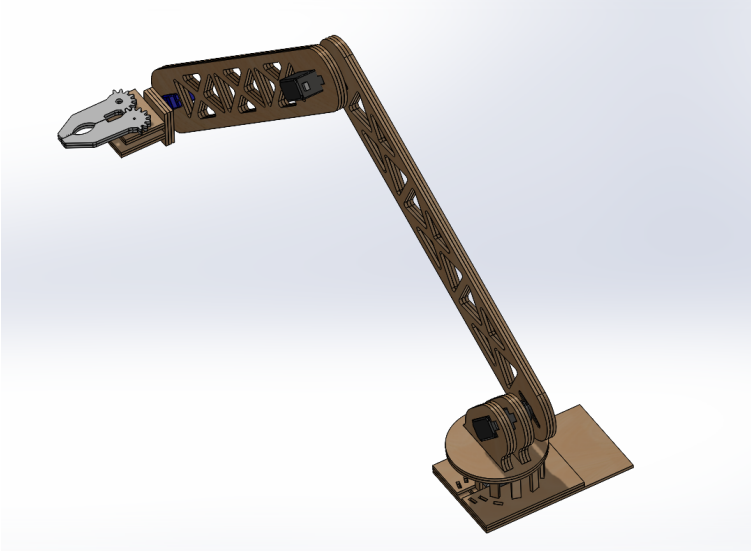


Figure 1: 3D CAD part of the robotic arm

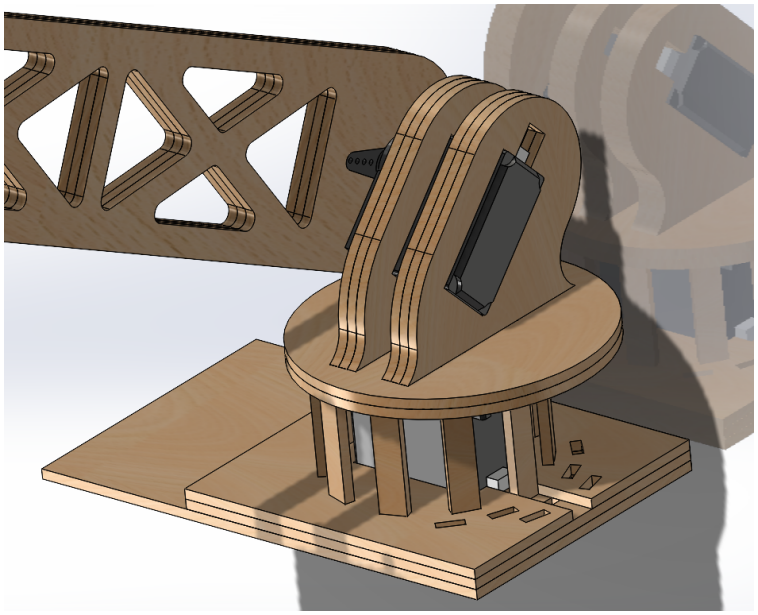


Figure 2: robotic arm base

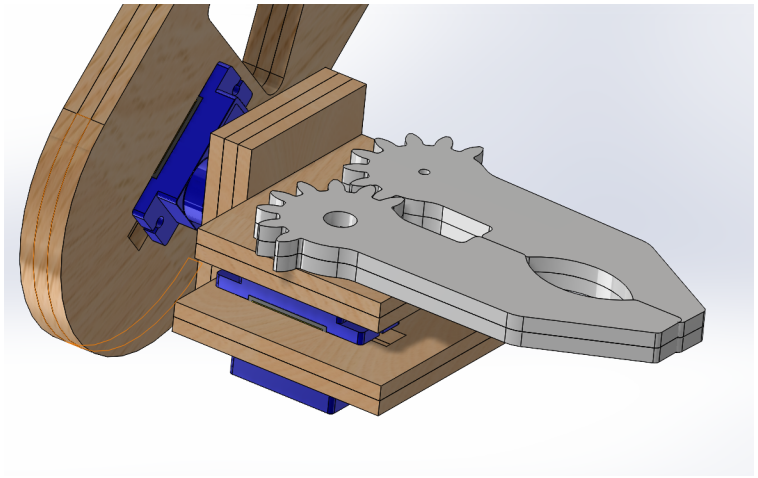


Figure 3: robotic arm end affector

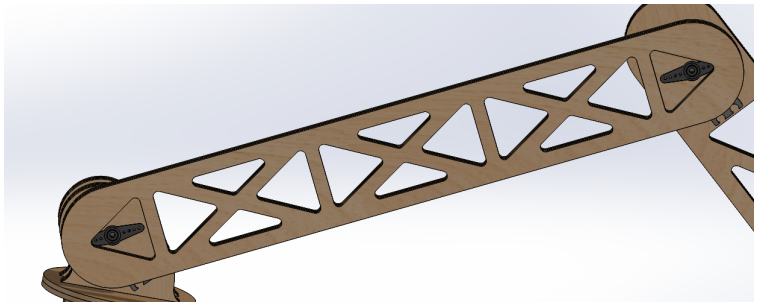


Figure 4: robotic arm first link

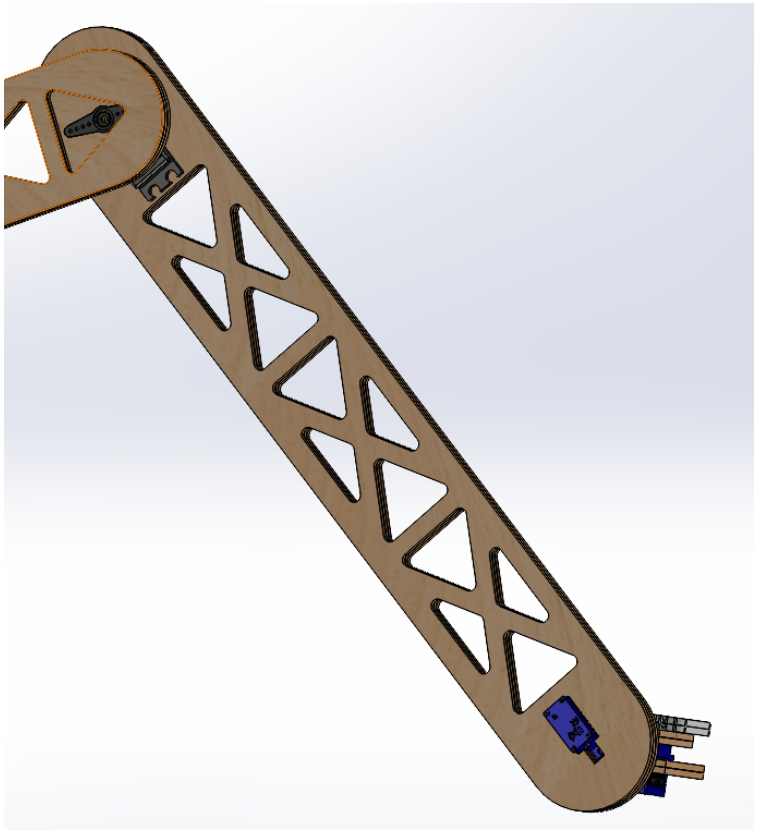


Figure 5: robotic arm second link

8 Proteus Circuit Simulation

To validate the robotic arm control system before hardware implementation, a Proteus simulation was created using an Arduino UNO and multiple servo motors. The main objective of the simulation is to verify the wiring connections, confirm correct PWM signal routing from the Arduino pins to each servo motor, and ensure that the power and grounding scheme is consistent for stable operation.

In the Proteus circuit, the Arduino UNO provides the control signals to the servo motors, where each servo represents a robotic joint (base, shoulder, elbow, wrist, and gripper). A common ground is used to avoid unstable behavior, while the external power supply is connected to provide sufficient current for the servo motors during motion.

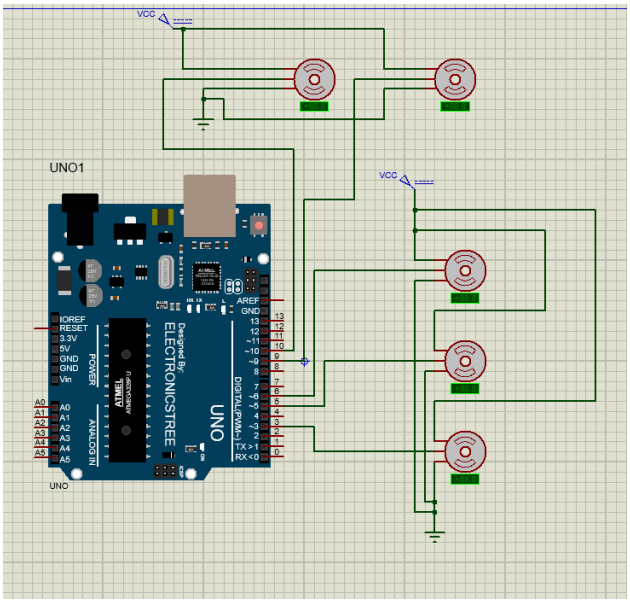


Figure 6: Proteus circuit schematic of the robotic arm.

9 Appendix A: Arduino Implementation of Inverse Kinematics

This appendix presents the Arduino source code used to implement the **inverse kinematics**–based control of the robotic arm. In this implementation, the desired end-effector position is specified in three-dimensional Cartesian space, and the corresponding joint angles are computed analytically.

The inverse kinematics algorithm calculates the base joint angle using trigonometric relations in the horizontal plane, while the shoulder and elbow joint angles are determined using geometric relationships and the law of cosines. The computed joint angles are then applied to the servo motors to position the robotic arm at the desired target location.

This method is particularly useful for point-to-point motion control where the end-effector position is defined in Cartesian coordinates.

9.1 Inverse Kinematics Arduino Code

Listing 1: Arduino code implementing inverse kinematics for the robotic arm

```
#include <Servo.h>
#include <math.h>

Servo base, shoulder, elbow, wrist, gripper;

// Link lengths (consistent units)
const float L1 = 35.0;
const float L2 = 35.0;
const float baseHeight = 5.0; // Base height offset

void setup() {
  base.attach(3);
  shoulder.attach(5);
  elbow.attach(6);
  wrist.attach(9);
  gripper.attach(10);

  Serial.begin(9600);

  // Move to a target position (X, Y, Z)
  moveArm3DIK(5, 20, 40);
}

void loop() {}

void moveArm3DIK(float x, float y, float z) {
  // Base angle
```

```
  float baseRad = atan2(y, x);
  int baseDeg = baseRad * 180.0 / PI;

  // Horizontal distance from base
  float r = sqrt(x*x + y*y);

  // Height relative to shoulder joint
  float relativeZ = z - baseHeight;

  // Distance from shoulder to wrist
  float dist = sqrt(r*r + relativeZ*relativeZ);

  // Reachability check
  if (dist >= (L1 + L2) || dist <= abs(L1 - L2)) {
    Serial.println("Target out of reach!");
    return;
  }

  // Elbow angle (law of cosines)
  float cosQ2 = (r*r + relativeZ*relativeZ - L1*L1 - L2*L2) / (2 * L1 * L2);
  float q2 = acos(cosQ2);

  // Shoulder angle
  float q1 = atan2(relativeZ, r) +
             atan2(L2 * sin(q2), L1 + L2 * cos(q2));

  // Convert to degrees
  int shoulderDeg = q1 * 180.0 / PI;
  int elbowDeg = q2 * 180.0 / PI;

  // Servo commands
  base.write(90 + baseDeg);
  shoulder.write(180 - shoulderDeg);
  elbow.write(elbowDeg);

  Serial.print("Base: "); Serial.print(90 + baseDeg);
  Serial.print(" | Shoulder: "); Serial.print(shoulderDeg);
  Serial.print(" | Elbow: "); Serial.println(elbowDeg);
}
```

10 Appendix B: Arduino Implementation of Forward Kinematics

This appendix presents the Arduino source code used to implement the **forward kinematics** control of the robotic arm. In this approach, the joint angles of the base, shoulder, elbow, wrist, and gripper are directly commanded in joint space, and the resulting end-effector motion follows from the known kinematic structure of the robot.

Unlike inverse kinematics, where joint angles are computed from a desired Cartesian position, this forward kinematics implementation executes predefined joint trajectories. Smooth motion is achieved by incrementally updating the servo angles, allowing coordinated and continuous movement of all joints.

10.1 Forward Kinematics Arduino Code

Listing 2: Arduino code implementing forward kinematics and smooth joint-space motion control

```
#include <Servo.h>

// ===== Servo Objects =====
Servo baseServo, shoulderServo, elbowServo, wristServo, gripperServo;

// ===== Pins =====
const int basePin     = 3;
const int shoulderPin = 5;
const int elbowPin    = 6;
const int wristPin    = 9;
const int gripperPin  = 10;

// ===== Link Lengths (cm) =====
float L1 = 5.0;    // base height
float L2 = 35.0;   // upper arm
float L3 = 35.0;   // forearm

// ===== Global Position Tracking =====
float curBase = 0, curShoulder = 0, curElbow = 0, curWrist = 0, curGripper =

void setup() {
  Serial.begin(9600);

  // Attach servos
  baseServo.attach(basePin);
  shoulderServo.attach(shoulderPin);
  elbowServo.attach(elbowPin);
  wristServo.attach(wristPin);
```

```
  gripperServo.attach(gripperPin);

  // Initialize all joints to home position
  baseServo.write(90);
  shoulderServo.write(90);
  elbowServo.write(90);
  wristServo.write(90);
  gripperServo.write(90);
  delay(2000);

  // ===== Motion Sequences =====
  moveArmSmoothly(45, -50, 20, 40, 50);
  moveGripperSmoothly(-25, 30);
  delay(2000);

  moveArmSmoothly(0, 0, 0, 0, 50);
  moveGripperSmoothly(0, 30);

  moveArmSmoothly(30, 45, 60, 50, 50);
  moveGripperSmoothly(-25, 30);
  delay(2000);

  moveArmSmoothly(0, 0, 0, 0, 50);
  moveGripperSmoothly(0, 30);
}

void loop() {
  // Runs once in setup
}

// ===== Function: Move Arm Joints Simultaneously =====
void moveArmSmoothly(float tBase, float tSh, float tEl, float tWr, int stepDe
  while (curBase != tBase || curShoulder != tSh || curElbow != tEl || curWris

    if (curBase < tBase) curBase++; else if (curBase > tBase) curBase--;
    if (curShoulder < tSh) curShoulder++; else if (curShoulder > tSh) curShou
    if (curElbow < tEl) curElbow++; else if (curElbow > tEl) curElbow--;
    if (curWrist < tWr) curWrist++; else if (curWrist > tWr) curWrist--;

    baseServo.write(90 + curBase);
    shoulderServo.write(90 + curShoulder);
    elbowServo.write(90 + curElbow);
    wristServo.write(90 + curWrist);

    delay(stepDelay);
```

```
  }
}

// ===== Function: Move Gripper Individually =====
void moveGripperSmoothly(float tGr, int stepDelay) {
  while (curGripper != tGr) {
    if (curGripper < tGr) curGripper++; else if (curGripper > tGr) curGripper
    gripperServo.write(90 + curGripper);
    delay(stepDelay);
  }
}
```

11 Appendix C: Algorithmic Verification Code

This appendix presents the MATLAB script used for **algorithmic verification** of the robotic arm kinematics. The purpose of this code is to validate the analytical forward and inverse kinematics derivations by performing a closed-loop consistency check between joint space and Cartesian space.

The verification process follows three main steps. First, a set of known joint angles is applied to the forward kinematics equations to compute the end-effector position. Second, the inverse kinematics algorithm is applied to the obtained position to recover the joint angles. Finally, the recovered joint angles are substituted back into the forward kinematics equations, and the resulting position and angle errors are evaluated.

This approach ensures the correctness, numerical stability, and consistency of the derived kinematic models before implementation on the Arduino-based control system.

11.1 MATLAB Algorithmic Verification Script

Listing 3: MATLAB code for algorithmic verification of forward and inverse kinematics

```
clc;
clear;
close all;

% PARAMETERS
L1 = 0.35;
L2 = 0.35;
H1 = 0.05;

% TEST JOINT ANGLES
theta1_test = deg2rad(80);
theta2_test = deg2rad(-15);
theta3_test = deg2rad(45);

fprintf('Test Joint Angles:\n');
fprintf('theta1 = %.2f \n', rad2deg(theta1_test));
fprintf('theta2 = %.2f \n', rad2deg(theta2_test));
fprintf('theta3 = %.2f \n\n', rad2deg(theta3_test));

% FORWARD KINEMATICS
fprintf('=== FORWARD KINEMATICS ===\n');

c1 = cos(theta1_test); s1 = sin(theta1_test);
c2 = cos(theta2_test); s2 = sin(theta2_test);
c3 = cos(theta3_test); s3 = sin(theta3_test);

A1 = [ c1   0   s1   0;
```

```
        s1   0 -c1   0;
        0    1   0    H1;
        0    0   0    1 ];

A2 = [ c2 -s2   0   L1*c2;
        s2   c2   0   L1*s2;
        0    0    1   0;
        0    0    0   1 ];

A3 = [ c3 -s3   0   L2*c3;
        s3   c3   0   L2*s3;
        0    0    1   0;
        0    0    0   1 ];

T03 = A1 * A2 * A3;

Px = T03(1,4);
Py = T03(2,4);
Pz = T03(3,4);

fprintf('End-effector position:\n');
fprintf('X = %.4f m\n', Px);
fprintf('Y = %.4f m\n', Py);
fprintf('Z = %.4f m\n\n', Pz);

% INVERSE KINEMATICS
fprintf('=== INVERSE KINEMATICS ===\n');

x = Px; y = Py; z = Pz;

theta1 = atan2(y, x);
x_proj = sqrt(x^2 + y^2);
z_proj = z - H1;

r = sqrt(x_proj^2 + z_proj^2);

cos_theta3 = (r^2 - L1^2 - L2^2) / (2 * L1 * L2);
cos_theta3 = min(max(cos_theta3, -1), 1);

theta3 = acos(cos_theta3);
beta = atan2(L2 * sin(theta3), L1 + L2 * cos(theta3));
theta2 = atan2(z_proj, x_proj) - beta;

% VERIFICATION
c1 = cos(theta1); s1 = sin(theta1);
```

```
c2 = cos(theta2); s2 = sin(theta2);
c3 = cos(theta3); s3 = sin(theta3);

A1 = [ c1   0   s1   0;
       s1   0 -c1   0;
       0    1   0   H1;
       0    0   0   1 ];

A2 = [ c2 -s2   0   L1*c2;
       s2   c2   0   L1*s2;
       0    0    1   0;
       0    0    0   1 ];

A3 = [ c3 -s3   0   L2*c3;
       s3   c3   0   L2*s3;
       0    0    1   0;
       0    0    0   1 ];

T03_ver = A1 * A2 * A3;

error_pos = norm(T03(1:3,4) - T03_ver(1:3,4));

fprintf('Position error = %.6f m\n', error_pos);

if error_pos < 1e-3
    fprintf('    Algorithmic verification successful.\n');
else
    fprintf('    Verification error detected.\n');
end
```